

Primer Parcial de Programación Orientada a Objetos (72.33)

18/04/2024

Ejercicio 1 (3.5 puntos)

Se desea modelar un sistema para la **gestión de puertos de ferrys** contemplando los amarres y desamarres de las embarcaciones en los muelles de cada uno de los puertos.

Ya cuenta con las clases **FerryCompany** y **Ferry** (ambas pueden modificarse) donde la primera corresponde a una compañía de ferrys (que consiste en un nombre) y la segunda corresponde a un ferry (que consiste en un nombre y una compañía de ferrys).

```
public class FerryCompany {
    private final String name;
    public FerryCompany(String name) {
        this.name = name;
    }
    @Override
    public String toString() {
        return name;
    }
}
```

```
public class Ferry {
    private final String name;
    private final FerryCompany company;
    public Ferry(String name, FerryCompany c) {
        this.name = name;
        this.company = c;
    }
    public FerryCompany getCompany() {
        return company;
    }
    @Override
    public String toString() {
        return "Ferry %s".formatted(name);
    }
}
```

Para instanciar un puerto se indica el **nombre** del mismo, la **compañía** a la cual pertenece y la **cantidad de muelles**. Todos los puertos deben contar con métodos para **amarrar (dock)** y **desamarrar (undock)** una embarcación. En ambos se indica el ferry involucrado y las horas y minutos de la operación.

En un puerto se pueden **amarrar** ferrys siempre y **cuando existan muelles libres** (con un amarre se ocupa un muelle) y el ferry sea de la **misma compañía** del puerto. Para simplificar el ejercicio no corresponde verificar que un ferry amarrado no pueda volver a amarrarse.

En un puerto se pueden **desamarrar** ferrys siempre y **cuando existan muelles ocupados** (con un desamarre se desocupa un muelle) y los ferrys pueden ser **de cualquier compañía**. Para simplificar el ejercicio no corresponde verificar que un ferry tuvo que ser amarrado antes de poder ser desamarrado.

Existen tres tipos de puertos:

- Un puerto que permite el **amarre las 24hs** del día
- Un puerto que sólo permite el **amarre** respetando los **horarios de apertura y cierre** del mismo (que se reciben por parámetro)
- Un puerto que sólo permite el **amarre** respetando los **horarios de apertura y cierre** del mismo (que se reciben por parámetro) y donde sólo se puede **desamarrar el último ferry amarrado** (si es que se amarró uno)

Para simplificar el ejercicio no corresponde verificar que el horario de apertura es anterior al horario de cierre.

Completar los A B C, implementar todo lo necesario para que, con el siguiente programa de prueba, se obtenga la salida indicada en los comentarios:

```

public static void main(String[] args) {
    // Ejemplo de uso de la clase java.time.LocalDateTime
    System.out.println(LocalTime.of(12, 15).isAfter(LocalTime.of(11, 15))); // true
    System.out.println(LocalTime.of(12, 15).isBefore(LocalTime.of(11, 15))); // false

    // Se instancia una compañía de ferry TransLink
    FerryCompany transLink = new FerryCompany("TransLink");
    // Se instancia un ferry Beaver de la compañía TransLink
    Ferry beaver = new Ferry("Beaver", transLink);
    Ferry otter = new Ferry("Otter II", transLink);
    Ferry pacificBreeze = new Ferry("Pacific Breeze", transLink);

    // Se instancia un puerto Waterfront de la compañía TransLink con 2 muelles
    // donde se pueden amarrar ferrys las 24hs
    ..... waterfront = new .....("Waterfront", transLink, 2);
    // Beaver se amarra a un muelle del puerto Waterfront a las 09:00
    waterfront.dock(beaver, LocalTime.of(9, 0));
    // Ferry Beaver docked at Port Waterfront
    // Beaver se desamarra de un muelle del puerto Waterfront a las 09:05
    waterfront.undock(beaver, LocalTime.of(9, 5));
    // Ferry Beaver undocked at Port Waterfront
    try { // Falla porque todos los muelles de Waterfront están desocupados
        waterfront.undock(beaver, LocalTime.of(10, 10));
    } catch (Exception ex) {
        System.out.println("Error"); // Error
    }
    waterfront.dock(beaver, LocalTime.of(10, 0));
    // Ferry Beaver docked at Port Waterfront
    waterfront.dock(otter, LocalTime.of(10, 15));
    // Ferry Otter II docked at Port Waterfront
    try { // Falla porque todos los muelles están ocupados
        waterfront.dock(pacificBreeze, LocalTime.of(10, 10));
    } catch (Exception ex) {
        System.out.println("Error"); // Error
    }
    // No corresponde verificar que un ferry tuvo que ser amarrado para ser desamarrado
    waterfront.undock(pacificBreeze, LocalTime.of(10, 20));
    // Ferry Pacific Breeze undocked at Port Waterfront
    try { // Falla porque las compañías no coinciden
        waterfront.dock(new Ferry("Spirit of Vancouver", new FerryCompany("BC Ferries")),
            LocalTime.of(10, 10));
    } catch (Exception ex) {
        System.out.println("Error"); // Error
    }
    // Se desamarra un ferry de una compañía distinta a la del puerto
    waterfront.undock(new Ferry("Spirit of Vancouver", new FerryCompany("BC Ferries")),
        LocalTime.of(10, 20));
    // Ferry Spirit of Vancouver undocked at Port Waterfront
    // No corresponde verificar que un ferry amarrado no puede volver a amarrarse
    waterfront.dock(beaver, LocalTime.of(10, 0));
    // Ferry Beaver docked at Port Waterfront

    // Se instancia un puerto Lonsdale Quay de la compañía Translink con 3 muelles
    // donde se pueden amarrar ferrys entre las 09:00 y las 14:00
    ..... lonsdaleQuay = new .....("Lonsdale Quay",
        new FerryCompany("TransLink"), 3, LocalTime.of(9, 0), LocalTime.of(14, 0));
    lonsdaleQuay.dock(beaver, LocalTime.of(9, 0));
    // Ferry Beaver docked at Port Lonsdale Quay
    lonsdaleQuay.dock(otter, LocalTime.of(12, 15));
    // Ferry Otter II docked at Port Lonsdale Quay

```

```

try { // Falla porque no respeta los horarios de apertura y cierre
    lonsdaleQuay.dock(beaver, LocalDateTime.of(15, 10));
} catch (Exception ex) {
    System.out.println("Error"); // Error
}
lonsdaleQuay.dock(beaver, LocalDateTime.of(14, 0));
// Ferry Beaver docked at Port Lonsdale Quay
// Se desamarra un ferry fuera de los horarios de apertura y cierre para los amarres
lonsdaleQuay.undock(beaver, LocalDateTime.of(23, 0));
// Ferry Beaver undocked at Port Lonsdale Quay

// Se instancia un puerto Stanley Park de la compañía Translink con 3 muelles
// donde se pueden amarrar ferrys entre las 09:00 y las 14:00
// y sólo se puede desamarrar el último ferry amarrado (si es que se amarró uno)
..... stanleyPark = new .....("Stanley Park", transLink, 3,
    LocalDateTime.of(9, 0), LocalDateTime.of(14, 0));
try { // Falla porque no se amarró ningún ferry
    stanleyPark.undock(beaver, LocalDateTime.of(15, 10));
} catch (Exception ex) {
    System.out.println("Error"); // Error
}
stanleyPark.dock(beaver, LocalDateTime.of(12, 10));
// Ferry Beaver docked at Port Stanley Park
stanleyPark.dock(otter, LocalDateTime.of(12, 15));
// Ferry Otter II docked at Port Stanley Park
try { // Falla porque el ferry a desamarrar no es el último amarrado
    stanleyPark.undock(beaver, LocalDateTime.of(15, 10));
} catch (Exception ex) {
    System.out.println("Error"); // Error
}
stanleyPark.undock(otter, LocalDateTime.of(13, 10));
// Ferry Otter II undocked at Port Stanley Park
try { // Falla porque no se amarró ningún ferry después de desamarrar uno
    stanleyPark.undock(beaver, LocalDateTime.of(15, 10));
} catch (Exception ex) {
    System.out.println("Error"); // Error
}
}

```



Ejercicio 2 (3 puntos)

Se desea modelar un **plan de pagos** que permita consultar el número de cuota y el monto a pagar en cada una de las cuotas.

Ya cuenta con la clase **PaymentDetail** (que no puede modificarse) que corresponde a un **detalle del plan de pagos** (una cuota en particular), indicando el número de cuota y el monto correspondiente.

```

public class PaymentDetail {
    private final int installment;
    private final double installmentAmount;
    public PaymentDetail(int installment, double installmentAmount) {
        this.installment = installment;
        this.installmentAmount = installmentAmount;
    }
    @Override
    public String toString() {
        return "%.2f due for installment %d".formatted(installmentAmount, installment);
    }
}

```

Para instanciar un plan de pagos se indica el **monto total** a pagar, la **cantidad de cuotas** (de forma que el valor a pagar de cada cuota será el resultado de dividir el monto total por la cantidad de cuotas) y el **tipo de plan** de pagos (que permitirá bonificaciones según las condiciones de cada tipo de plan).

Existen tres tipos de planes de pago:

- Un plan de pagos donde la **cantidad máxima de cuotas es 12** y **no se aplican bonificaciones**
- Un plan de pagos donde la **cantidad máxima de cuotas es 24** y **se bonifican al 100% las primeras tres cuotas** (Para simplificar el ejercicio no corresponde validar una cantidad mínima de cuotas, si se eligen tres o menos cuotas la bonificación se aplica igual y todas las cuotas serán de \$0)
- Un plan de pagos donde la **cantidad máxima de cuotas es 360** y **se bonifican al 50% las cuotas pares** (la cuota 2, 4, etc.)

Al instanciar un plan de pagos se debe arrojar un error si se supera la cantidad máxima de cuotas del tipo de plan indicado o si el monto o la cantidad de cuotas no es un valor positivo.

Para consultar el detalle de la próxima cuota del plan de pagos existen dos formas:

- **next** consulta la próxima cuota de forma que una segunda invocación a **next** consulta una cuota siguiente
- **peek** consulta la próxima cuota de forma que una segunda invocación a **peek** consulta la misma cuota

Completar losD.....E.....F....., implementar PaymentPlan, InvalidPaymentPlanException y todo lo necesario para que, con el siguiente programa de prueba, se obtenga la salida indicada en los comentarios:

```
public static void main(String[] args) {
    // Se instancia un plan de pagos de hasta 12 cuotas sin bonificaciones
    // para $3.000,00 en 3 cuotas
    PaymentPlan pp1 = new PaymentPlan(....., 3_000, 3);
    // Se consulta el detalle de la siguiente cuota con next
    System.out.println(pp1.next()); // $1000,00 due for installment 1
    System.out.println(pp1.next()); // $1000,00 due for installment 2
    System.out.println(pp1.next()); // $1000,00 due for installment 3
    try { // Falla porque ya se consultaron todas las cuotas con next
        pp1.next();
    } catch (Exception ex) {
        System.out.println("No more installments"); // No more installments
    }

    // Se instancia un plan de pagos de hasta 24 cuotas con las 3 primeras gratis
    // para $5.000,00 en 5 cuotas
    PaymentPlan pp2 = new PaymentPlan(....., 5_000, 5);
    // Se consulta el detalle de la siguiente cuota con peek
    System.out.println(pp2.peek()); // $0,00 due for installment 1
    System.out.println(pp2.next()); // $0,00 due for installment 1
    System.out.println(pp2.next()); // $0,00 due for installment 2
    System.out.println(pp2.peek()); // $0,00 due for installment 3
    System.out.println(pp2.peek()); // $0,00 due for installment 3
    System.out.println(pp2.next()); // $0,00 due for installment 3
    System.out.println(pp2.next()); // $1000,00 due for installment 4
    System.out.println(pp2.next()); // $1000,00 due for installment 5
    try { // Falla porque ya se consultaron todas las cuotas con next
        pp2.peek();
    } catch (Exception ex) {
        System.out.println("No more installments"); // No more installments
    }
}
```

```

// Se instancia un plan de pagos de hasta 360 cuotas con las pares al 50%
// para $6.000,00 en 5 cuotas
PaymentPlan pp3 = new PaymentPlan(....., 6_000, 5);
while(pp3.hasNext()) {
    System.out.println(pp3.next());
}
// $1200,00 due for installment 1
// $600,00 due for installment 2
// $1200,00 due for installment 3
// $600,00 due for installment 4
// $1200,00 due for installment 5

try { // Falla pues la cantidad de cuotas supera el máximo permitido
    new PaymentPlan(.....D....., 3_000, 13);
} catch (InvalidPaymentPlanException ex) {
    System.out.println(ex.getMessage()); // Invalid payment plan
}
try { // Falla pues la cantidad de cuotas no es positiva
    new PaymentPlan(.....D....., 3_000, -2);
} catch (InvalidPaymentPlanException ex) {
    System.out.println(ex.getMessage()); // Invalid payment plan
}
try { // Falla pues el monto no es positivo
    new PaymentPlan(.....D....., 0, 12);
} catch (InvalidPaymentPlanException ex) {
    System.out.println(ex.getMessage()); // Invalid payment plan
}
}
}

```



Ejercicio 3 (3.5 puntos)

Se desea modelar un sistema para la **gestión de mostradores de check-in de un aeropuerto**. El mismo debe contar con un método que agrega un mostrador al aeropuerto (respetando una numeración incremental que inicia en 0) indicando qué aerolínea lo utilizará. Todos los mostradores se inician libres (*idle*), esto es, no están siendo usados para check-in.

Para iniciar el check-in en un mostrador existen dos formas:

- **checkIn** sobre un **mostrador**
- **counterCheckIn** sobre un **aeropuerto**, donde se inicia el **check-in en el primer mostrador (en orden de inserción)** que cumpla una condición enviada por parámetro. Si no existen mostradores en el aeropuerto o ninguno cumple la condición la operación no tiene efecto.

Para simplificar el ejercicio, si se intenta realizar el check-in (en cualquiera de las dos formas) en un mostrador que ya estaba realizando check-in la operación no tiene efecto.

Para consultar los mostradores existen dos formas:

- **airlineOrderCounters** retorna los mostradores en **orden alfabético por aerolínea y desempata ascendente por número**
- **airlineDescdeningOrderCounters** retorna los mostradores en **orden descendente por aerolínea y desempata descendente por número**

Completar elG....., implementar Airport, Counter y todo lo necesario para que, con el siguiente programa de prueba, se obtenga la salida indicada en los comentarios:

```

public static void main(String[] args) {
    // Se instancia un aeropuerto
    AirPort airPort = new AirPort();

    // Se instancia el mostrador 0 para Air Canada e inicia libre
    Counter counter0 = airPort.addCounter("Air Canada");
    System.out.println(counter0); // Counter 0 for Air Canada is idle
    Counter counter1 = airPort.addCounter("American Airlines");
    System.out.println(counter1); // Counter 1 for American Airlines is idle
    Counter counter2 = airPort.addCounter("American Airlines");
    System.out.println(counter2); // Counter 2 for American Airlines is idle
    airPort.addCounter("Air Canada");
    airPort.addCounter("LATAM");

    // Se inicia el check-in en el mostrador 1
    counter1.checkIn();
    System.out.println(counter1); // Counter 1 for American Airlines is checking in

    // Se obtienen los mostradores en orden alfabético por aerolínea
    // y desempata ascendente por número
    for(Counter counter : airPort.airLineOrderCounters()) {
        System.out.println(counter);
    }
    // Counter 0 for Air Canada is idle
    // Counter 3 for Air Canada is idle
    // Counter 1 for American Airlines is checking in
    // Counter 2 for American Airlines is idle
    // Counter 4 for LATAM is idle

    // Se obtienen los mostradores en orden descendente por aerolínea
    // y desempata descendente por número
    for(Counter counter : airPort.airLineDescendingOrderCounters()) {
        System.out.println(counter);
    }
    // Counter 4 for LATAM is idle
    // Counter 2 for American Airlines is idle
    // Counter 1 for American Airlines is checking in
    // Counter 3 for Air Canada is idle
    // Counter 0 for Air Canada is idle

    // Se inicia el check-in en el primer mostrador (en orden de inserción)
    // que cumpla las siguientes condiciones:
    // - No inició check-in
    // - La aerolínea empieza con la letra "A"
    // Si ningún mostrador cumple las condiciones no hace nada
    ..... idleAndStartsWithA = .....;
    airPort.counterCheckIn(idleAndStartsWithA);
    System.out.println(counter0); // Counter 0 for Air Canada is checking in
    System.out.println(counter1); // Counter 1 for American Airlines is checking in
    System.out.println(counter2); // Counter 2 for American Airlines is idle

    airPort.counterCheckIn(idleAndStartsWithA);
    System.out.println(counter0); // Counter 0 for Air Canada is checking in
    System.out.println(counter1); // Counter 1 for American Airlines is checking in
    System.out.println(counter2); // Counter 2 for American Airlines is checking in
}

```